

Programming II

Object Oriented Programming (OOP) **Inheritance and polymorphism**

Inheritance:

- Real life objects are typically specialized versions of other more general objects.

Example:

Without Inheritance

Employee

- Last name
- First name
- Address
- Home phone
- Hire date
- Pay grade

- +Update address()
- +Update pay grade()

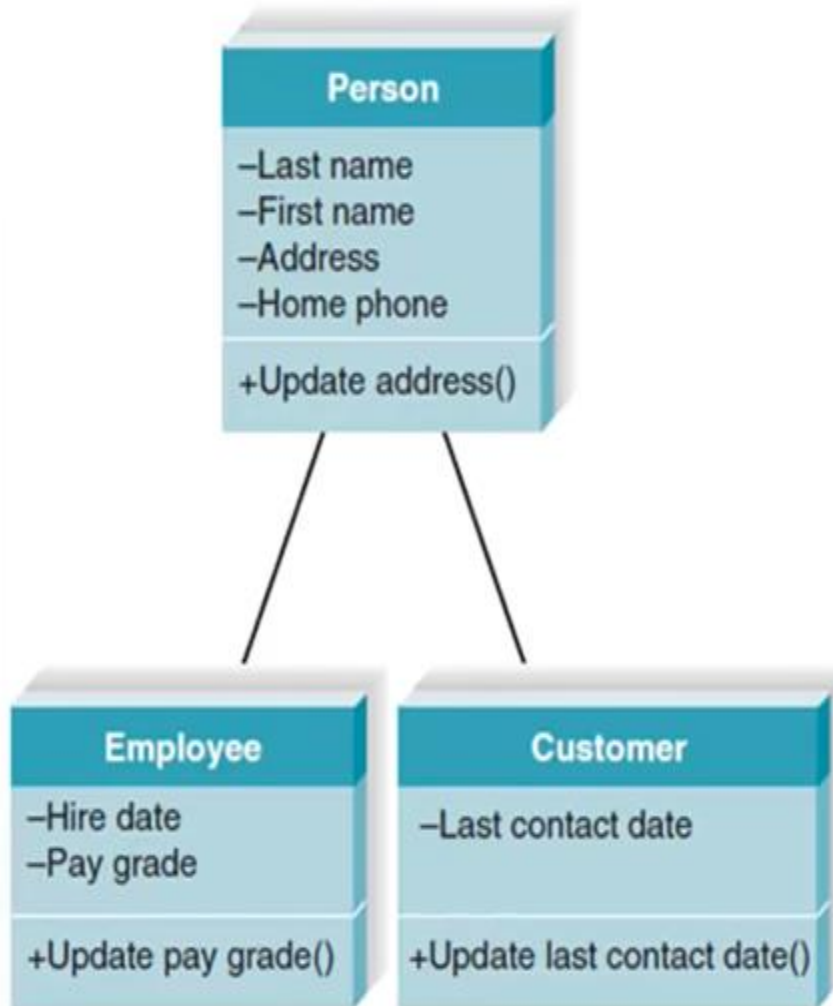
Customer

- Last name
- First name
- Address
- Home phone
- Last contact date

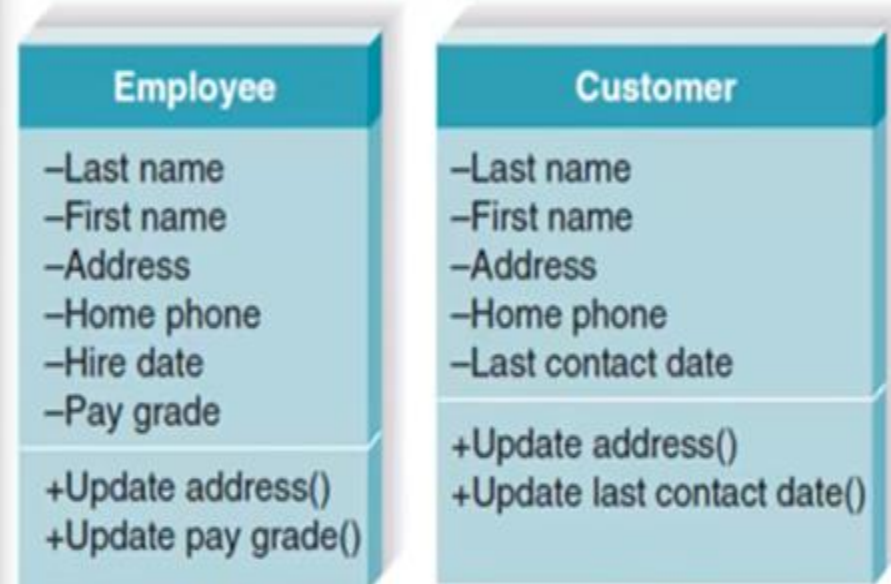
- +Update address()
- +Update last contact date()

Example:

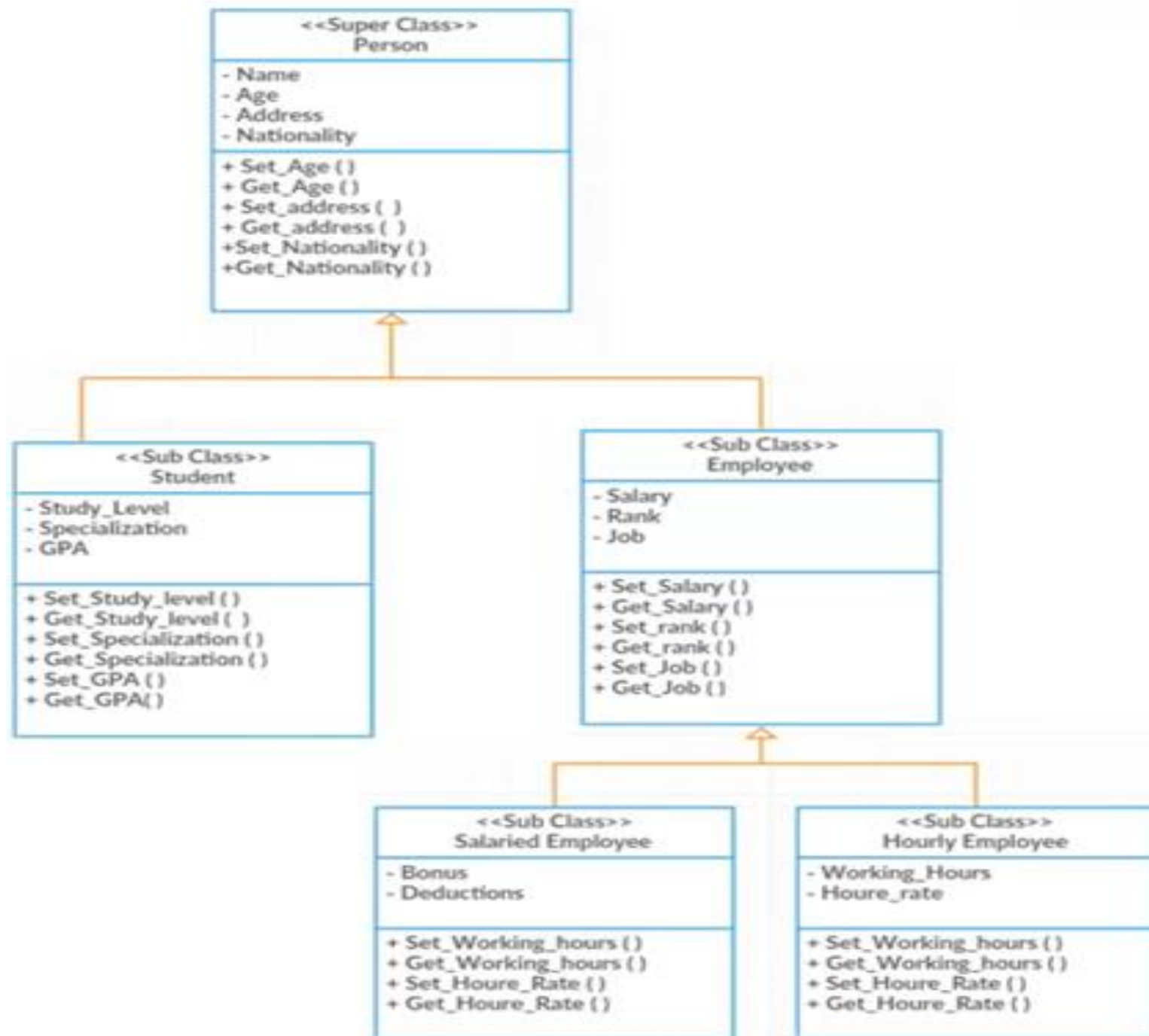
With Inheritance



Without Inheritance



Example:



The “is a “ Relationship:

- The relationship between a superclass and an inherited is called an “is a” relationship.
 - An Employee “is a” person.
 - Salaried employee “is an” Employee.
- A specialized object has:
 - All of the characteristics of the general object, plus additional characteristics that make it special.
- In object-oriented programming, inheritance is used to create an “is a” relationship among classes.

The “is a “ Relationship:

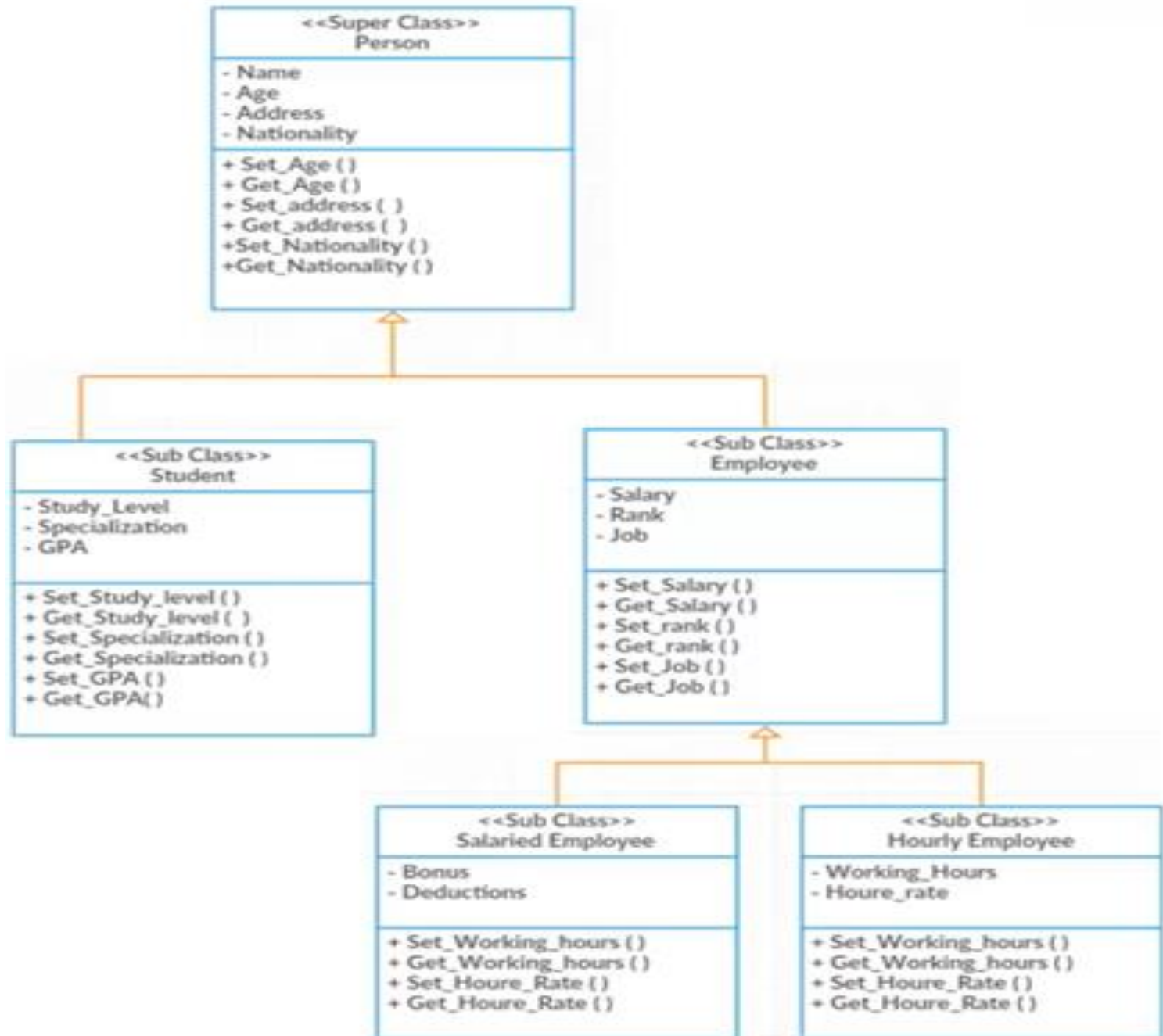
- We can extend the capabilities of a class.
- Inheritance involves a superclass and a subclass.
 - The superclass is the general class and the subclass is the specialized class.
- The subclass is based on, or extended from, the superclass.
 - Super classes are also called base classes, and subclasses are also called derived classes.
- The relationship of classes can be thought of as parent classes and child classes.

Inheritance:

- The subclass inherits fields and methods from the superclass without any of them being rewritten.
- New fields and methods may be added to the subclass.
- The Java keyword, **extends**, is used on the class header to define the subclass.

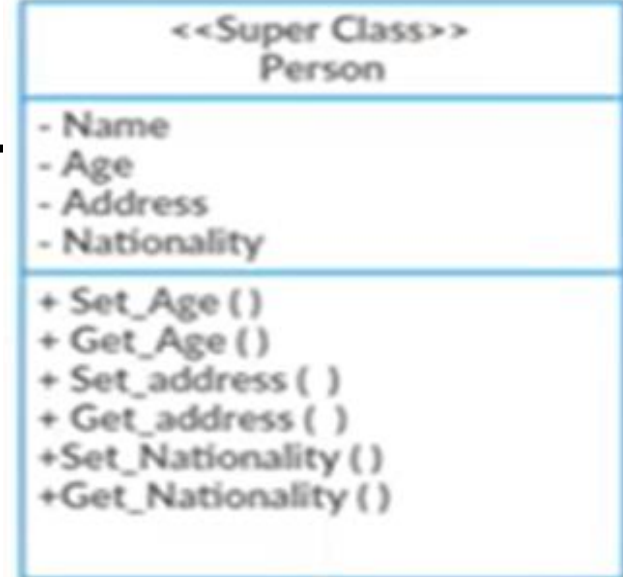
```
public class Employee extends person
```

Example:



Class person:

```
public class person
{ String name;
  double age;
  String address;
  String nationality;
  public person ()
  {
  }
  public person (String n, double a, String ad, String nat)
  { name = n;
    age = a;
    address = ad;
    nationality = nat;}
  public void set_name (String n){
    name = n;}
  public void set_age (double a){
    age = a;
  }
  public void set_address (String ad){
    address = ad;}
  public void set_nationality (String nat){
    nationality = nat;}
```



Class person:

```
public String get_name (){  
    return name;}  
public double get_age (){  
    return age;}  
public String get_address (){  
    return address;}  
public String get_nationality (){  
    return nationality;}  
}
```

Class student:

```
public class Student extends person
{
}

```

Test Program:

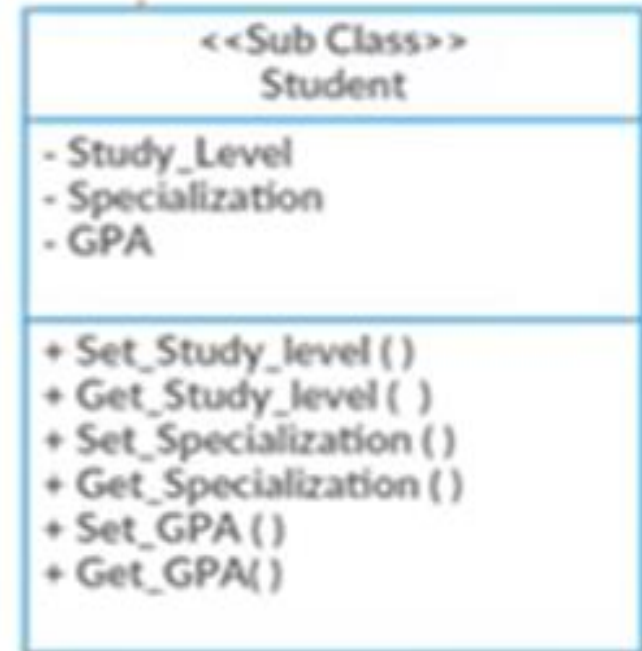
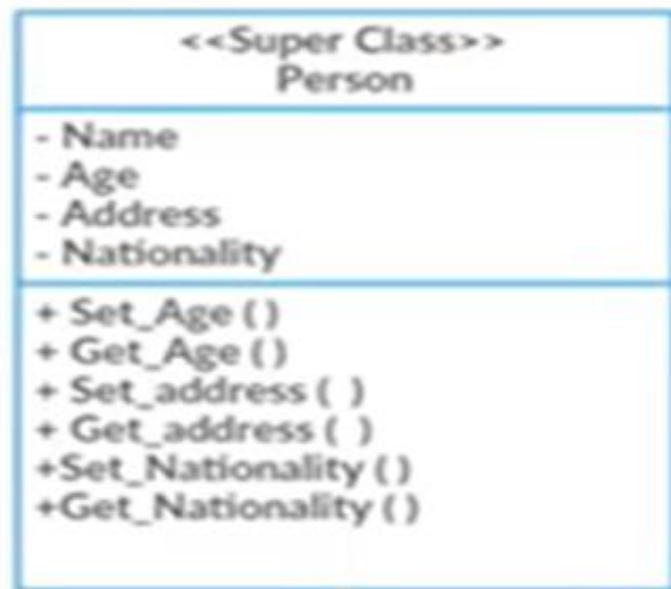
```
public class Main
{
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.set_age (25);
        System.out.println("The age is "+s1.get_age());
    }
}

```

The age is 25.0

Class student (Create the attributes)

```
public class Student extends person
{ private int study_level;
  private String Specialization;
  private double GPA;
  public Student ()
  {
  }
}
```



Class student (Create the constructor)

Super Keyword:

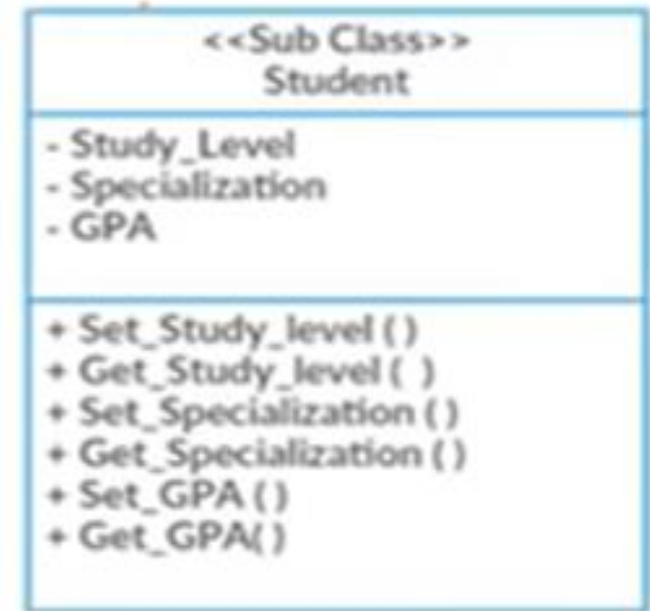
(reusability)

```
/*public Student (String n,double a , String ad, String nat, int lev, String special, double gpa)
{
    name = n;
    age = a;
    nationality = nat;
    address = ad;
    study_level = lev;
    Specialization = special;
    GPA = gpa;
} */
```

```
public Student (String n,double a , String ad, String nat, int lev, String special, double gpa)
{
    super (n,a,ad,nat);    // used to call the constructor of super class
    study_level = lev;
    Specialization = special;
    GPA = gpa;
}
}
```

Class Student (Complete Attributes and methods)

```
public class Student extends person
{ private int study_level;
  private String Specialization;
  private double GPA;
  public Student ()
  {
  }
}
```

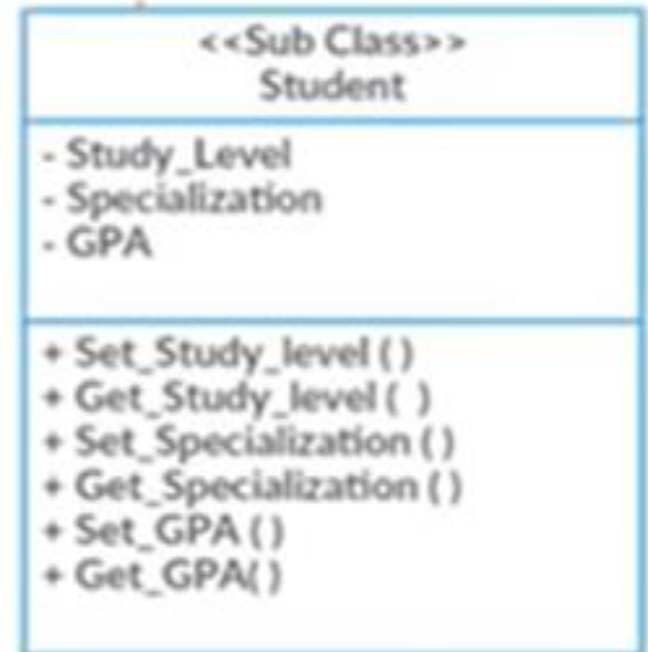


```
public Student (String n,double a , String ad, String nat, int lev, String special, double gpa)
{ super (n,a,ad,nat);    // used to call the constructor of super class
  study_level = lev;
  Specialization = special;
  GPA = gpa;
}
```

Class Student (Complete Attributes and methods)

```
public void set_GPA(double GPA)
{ //GPA = GPA;      (Confusion between attribute and argument)
  this.GPA = GPA;    //(this.GPA>>>attribute - GPA>>> argument)
}
public void set_studylevel(int study_level)
{ this.study_level = study_level;
}
public void set_GPA(String Specialization)
{ this.Specialization = Specialization;
}

public double get_GPA()
{ return GPA;
}
public int get_studylevel()
{ return study_level;
}
public String get_Specialization()
{ return Specialization;
}
}
```



Test Program:

```
public class Main
{
    public static void main(String[] args) {
        Student s1 = new Student("Ahmed", 20, "Alex", "Egyptian", 2, "AI", 3.2);
        System.out.println("The name is "+s1.get_name());
        System.out.println("The nationality is "+s1.get_nationality());
        System.out.println("The specialization is "+s1.get_Specialization());
    }
}
```

Output:

```
The name is Ahmed
The nationality is Egyptian
The specialization is AI
```

Inheritance, Fields and Methods:

- Members of the superclass that are marked private:
 - are not inherited by the subclass, and
 - may only be accessed from the subclass by public methods of the superclass.
- Members of the superclass that are marked public:
 - are inherited by the subclass, and
 - may be directly accessed from the subclass.

Inheritance and Constructors:

- Constructors are not inherited.
- The superclass constructor can be explicitly called from the subclass using the **super** keyword.
- When a subclass is instantiated (create object from subclass), the superclass default constructor is executed first.
 >> incase of object without any arguments

Note:

- When a subclass is instantiated (create object from subclass), the superclass default constructor is executed first.

Superclass:

```
public class person
{ String name;
  double age;
  String address;
  String nationality;
  public person () // default constructor without arguments
  { System.out.println ("Iam the Base Class Constructor");
  }
  public person (String n, double a, String ad, String nat)
  { name = n;
    age = a;
    address = ad;
    nationality = nat;}
```

Subclass:

```
public class Student extends person
{ private int study_level;
  private String Specialization;
  private double GPA;
  public Student ()
  {
    System.out.println ("Iam the Derived Class Constructor");
  }

  public Student (String n,double a , String ad, String nat, int lev, String special, double gpa)
  { super (n,a,ad,nat);      // used to call the constructor of super class
    study_level = lev;
    Specialization = special;
    GPA = gpa;
  }
}
```

Test program:

```
public class Main
{
  public static void main(String[] args) {
    Student s1 = new Student();

  }
}
```

Iam the Base Class Constructor
Iam the Derived Class Constructor

Test Program:

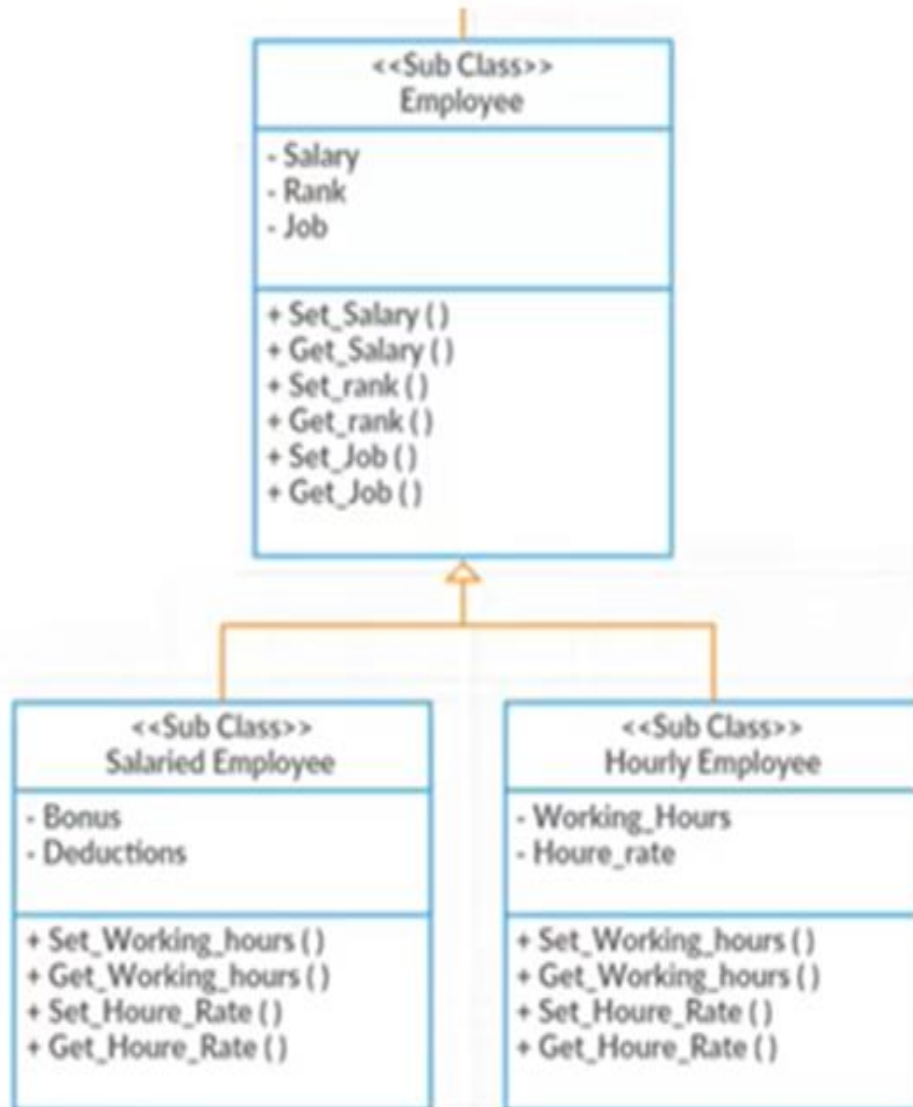
```
public class Main
{
    public static void main(String[] args) {
        Student s1 = new Student("Ahmed", 20, "Alex", "Egyptian", 2, "AI", 3.2);
        System.out.println("The name is "+s1.get_name());
        System.out.println("The nationality is "+s1.get_nationality());
        System.out.println("The specialization is "+s1.get_Specialization());
        Student s2 = new Student();
    }
}
```

```
The name is Ahmed
The nationality is Egyptian
The specialization is AI
Iam the Base Class Constructor
Iam the Derived Class Constructor
```

Overriding Superclass Methods:

- A subclass may have a method with the same signature as a superclass method.
- The subclass method overrides the superclass method.
- This is known as *method overriding*.
- A subclass method that overrides a superclass method must have the same signature as the superclass method.
- An object of the subclass invokes the subclass's version of the method, not the superclass's.
- The *@Override* annotation should be used just before the subclass method.

Example:



Employee

```
public double get_salary()
{
    return salary;
}
```

Salaried Employee

```
public double get_salary()
{
    return salary + bonus - deductions ;
}
```

Hourly Employee

```
public double get_salary()
{
    return working_hours * hours_rate ;
}
```

Example: Class Employee:

```
public class Employee extends person
{double salary;
  String rank;
  String Job;
public Employee()
{
}
public Employee (String n,double a , String ad, String nat, double sal, String r, String J)
{ super (n,a,ad,nat);
  salary = sal;
  rank = r;
  Job = J;
}
public void set_salary (double s)
{
  salary = s;
}
public double get_salary ()
{
  return salary;
}
}
```

Example: Class Salaried Employee:

```
public class Salaried_Employee extends Employee
{
    double bonous;
    double deduction;

    public Salaried_Employee()
    {
    }

    public Salaried_Employee (String n,double a , String ad, String nat, double sal, String r, String J, double b, double d)
    {
        super (n,a,ad,nat,sal,r,J);
        bonous = b;
        deduction = d;
    }

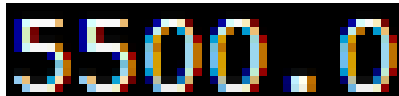
    @Override
    public double get_salary ()
    {
        return salary+bonous-deduction;
    }
}
```

Example:

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        Salaried_Employee sal1 = new Salaried_Employee ("Ahmed",25,"Alex","Egyptian",5000,"manager","Engineer",1000,500);
        System.out.println(sal1.get_salary());
    }
}
```

Output:



5500.0

Example:

Class Person:

```
public void print_all_details()
{
    System.out.println("Name = "+name+"\nAge = "+age+"\nAddress= "+address+"\nNationality = "+nationality);
}
```

Class Employee:

```
public void print_all_details()
{
    System.out.println("Name = "+name+"\nAge = "+age+"\nAddress= "+address+"\nNationality = "+nationality);
}
```

Example:

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        Employee E1 = new Employee ("Ahmed",25,"Alex","Egyptian",5000,"manager","Engineer");
        E1.print_all_details();
    }
}
```

Output:

```
Name = Ahmed
Age = 25.0
Address= Alex
Nationality = Egyptian
Job = Engineer
Rank = manager
Salary= 5000.0
```

Polymorphism:

Reference Type & Object Type:

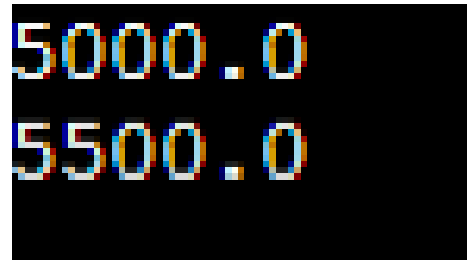
reference_type reference_variable = new object_type()

- You can create an object from a reference type superclass and an object type subclass.
- You can't create an object from a reference type subclass and an object type superclass.
- If you create an object from a reference type superclass and an object type subclass, you can use only the methods in the superclass.

Reference Type & Object Type:

```
public class Main
{
    public static void main(String[] args) {
        // Reference Type is Employee and Object Type is Employee
        Employee E1 = new Employee ("Ahmed",25,"Alex","Egyptian",5000,"manager","Engineer");
        // Reference Type is Employee and Object Type is Salaried_Employee
        Employee E2 = new Salaried_Employee ("Ahmed",25,"Alex","Egyptian",5000,"manager","Engineer",1000,500);
        System.out.println(E1.get_salary());
        System.out.println(E2.get_salary());
    }
}
```

Output:



```
5000.0
5500.0
```

Reference Type & Object Type:

```
public class Main
{
    public static void main(String[] args) {
        // Reference Type is Employee and Object Type is Employee
        Employee E1 = new Employee ("Ahmed",25,"Alex","Egyptian",5000,"manager","Engineer");
        // Reference Type is Salaried_Employee and Object Type is Employee
        Salaried_Employee E2 = new Employee ("Ahmed",25,"Alex","Egyptian",5000,"manager","Engineer");
    }
}
```

Output:

```
Main.java:15: error: incompatible types: Employee cannot be converted to Salaried_Employee
        Salaried_Employee E2 = new Employee ("Ahmed",25,"Alex","Egyptian",5000,"manager","Engineer");
                                   ^
1 error
```

Reference Type & Object Type:

Salaried Employee Class:

```
public void special_function()
{
}
}
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        // Reference Type is Employee and Object Type is Employee
        Employee E1 = new Employee ("Ahmed",25,"Alex","Egyptian",5000,"manager","Engineer");
        // Reference Type is Employee and Object Type is Salaried_Employee
        Employee E2 = new Salaried_Employee ("Ahmed",25,"Alex","Egyptian",5000,"manager","Engineer",1000,500);

        E2.special_function();
    }
}
```

Output:

```
Main.java:17: error: cannot find symbol
        E2.special_function();
            ^
    symbol:   method special_function()
    location: variable E2 of type Employee
1 error
```

Preventing a Method from Being Overridden:

- The final modifier will prevent the overriding of a superclass method in a subclass.

Public final void display()

- If a subclass attempts to override a final method, the compiler generates an error.
- This ensures that a particular superclass method is used by subclasses rather than a modified version of it.

Example:

Employee Class:

```
public final double get_salary ()  
{  
    return salary;  
}
```

Salaried Employee Class:

```
@Override  
public double get_salary ()  
{  
    return salary+bonous-deduction;  
}
```

Run from main method:

```
Salaried_Employee.java:16: error: get_salary() in Salaried_Employee cannot override get_salary() in Employee  
public double get_salary ()  
           ^  
    overridden method is final  
1 error
```

Protected Members:

- Using **protected** instead of private makes some tasks easier.
- Any class that is derived from the class, or is in the same package, has unrestricted access to the protected member.
- It is always better to make all fields **private** and then provide **public** methods for accessing those fields.
- Any method in the same package may access the protected member.

Example:

```
public class Shape
{
    private double height; // To hold height.
    private double width; //To hold width or base

    /**
     * The setValue method sets the data
     * in the height and width field.
     */
    public void setValues(double height, double width)
    {
        this.height = height;
        this.width = width;
    }
}
```

```
public class Rectangle extends Shape
{

    /**
     * The method returns the area
     * of rectangle.
     */
    public double getArea()
    {
        return height * width; //accessing protected members
    }
}
```

- **Generates error.**
- **The solution is protected member.**

Example:

```
public class Shape
{
    Protected double height; // To hold height.
    Protected double width; //To hold width or base

    /**
     * The setValue method sets the data
     * in the height and width field.
     */
    public void setValues(double height, double width)
    {
        this.height = height;
        this.width = width;
    }
}
```

```
public class Rectangle extends Shape
{

    /**
     * The method returns the area
     * of rectangle.
     */
    public double getArea()
    {
        return height * width; //accessing protected members
    }
}
```

Example:

```
public class Shape
{private double length;
  private double width;
  public Shape()
  {
  }
  public Shape (double length,double width)
  { this.length = length;
    this.width = width;
  }
}
```

```
Rectangle.java:8: error: length has private access in Shape
    return length*width;
           ^
```

```
Rectangle.java:8: error: width has private access in Shape
    return length*width;
           ^
```

2 errors

```
public class Rectangle extends Shape
{
    public Rectangle ()
    {
    }
    public double get_area ()
    {
        return length*width;
    }
}
```

Example:

```
public class Shape
{protected double length;
  protected double width;
  public Shape()
  {
  }
  public Shape (double length,double width)
  { this.length = length;
    this.width = width;
  }
}
```

Access Modifiers:

Modifier	Same class	Same Package	Subclass (different Package)	Other classes
Public	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	No
default	Yes	Yes	No	No
private	Yes	No	No	No